CBU5000 V210 Radar Driver (2.1)

RADAR TEST APP USER MANUAL

ShenZhen ChipsBank Technology Co., Ltd.

http://www.chipsbank.com/

7th Floor, Building 12, Phase II, Shenzhen Software Park,

Kejizhong 2nd Road, Nanshan District

Revision History

Version No	Description	Date	Review Date	Author
0.1	• Document Creation	14 Aug 2026	19 Aug 2026	Siegfred
0.2	• Change title, update formula	19 Aug 2026		Siegfred
0.3	• Add preamble code selection for BPRF and HPRF	20 Aug 2026		Siegfred
0.4	• Add preamble selection for LG4A • Update burst time based on 64 MHz clock • Shift the reference to index =30 (R=0)	25 Aug 2026	26 Aug 2026	Siegfred
0.5	• Fix formatting issues	31 Aug 2026		Siegfred

© ChipsBank 2026

Table of Content

1 INTRODUCTION 5

2 TEST APP PARAMETERS 5

2.1 MODE_PRF 6
2.2 POWER CODE, GAIN INDEX AND SCALE BIT 6
2.3 CIR_TAP_START AND CIR_TAP_END 7
2.4 CONTINUOUS SENSING (SENSING_MODE = 0) 7
2.5 BURST MODE SENSING_BURST_NUM 7
2.6 FRAME-BASED SENSING (SENSING_MODE = 1) 8

3 TEST APP DATA FORMAT 9

3.1 SENSING MODE = 0 (CONTINUOUS SENSING) 9
3.2 SENSING MODE = 1 (FRAME-BASED SENSING) 10
3.3 LOGGING IN BINARY FORMAT 12
3.4 TEST APP COMMAND 13

© ChipsBank 2026

List of Figures

Figure 2-1. Illustration for the components being controlled by the power code, gain index and scale bit. 6
Figure 2-2. Continuous sensing mode 7
Figure 2-3. Continuous sensing mode with burst. 7
Figure 2-4. Frame-based mode. Burst mode can be enabled but not shown in this figure. 8
Figure 3-1. Test app timing diagram for continuous mode. $\boldsymbol{TS}$, $\boldsymbol{TD}$ and $\boldsymbol{Ti}$ are the sensing, data transfer and idle time for each CIR cycle, respectively. 10
Figure 3-2. Test app timing diagram for frame-based mode. $\boldsymbol{TS}$, $\boldsymbol{TD}$ and $\boldsymbol{Ti}$ are the sensing, data transfer and idle time for one complete frame, respectively. 11
Figure 3-3. CIR data returned by the test App via UART. 12

© ChipsBank 2026

List of Tables

Table 1. Parameters for the test app 5
Table 2. Different PRF modes used in radar sensing 6
Table 3. Power code and corresponding power density 6
Table 4. Gain index and the corresponding gain, noise figure and input power saturation point 7
Table 5. Maximum number of sensing_burst_num for $\mathrm{num_rx_mode}=1$ 8
Table 6. Maximum number of sensing_burst_num for $\mathrm{num_rx_mode}=2$ 8
Table 7. An example of parameter settings for continuous mode 9
Table 8. An example of parameter settings for frame-based mode 10

© ChipsBank 2026

1 Introduction

This document gives details of the current radar driver release and the accompanying test app. The test app demonstrates two modes of operation – continuous-sensing mode, and frame-based mode. The primary purpose of the app is to demonstrate the use of the radar driver for different frame format, retrieve the CIR data and send over the UART.

2 Test App Parameters

The list of radar parameters are given below. In the command-line version of the test app, twelve parameters must be provided. Table 1 shows the table of the parameters, with the details given in the subsequent subsections.

Table 1. Parameters for the test app

idx	Radar Config Parameter, P[idx]	Description
0	**mode_PRF**	**0** – BPRF **1** – HPRF, **2** – LG4A
1	**power_code**	Power code **0-60**
2	**scale_bit**	Receiver scale bit. Recommended values: **4** – LG4A **6** – BPRF and HPRF
3	**gain_idx**	**0-7** Receiver gain index
4	**CIR_tap_start**	Starting index. Minimum value is 0. CIR_Tap_start = 30 is the assumed R = 0 m.
5	**CIR_tap_end**	Ending index. Total CIR taps returned is (CIR_Tap_end – CIR_Tap_start + 1) CIR_tap_end maximum value is 235 corresponding to the maximum range R = 30.8 m. Each tap is approximately 15.024 cm.
6	**sensing_mode**	**0** – if continuous mode **1** – if frame-based mode
7	**sensing_burst_num**	**1-256** but the actual value to be calculated at the back-end
8	**sensing_interval_ms**	Interval of the CIR in slow-time. Minimum value is 1. Maximum value is 10000 (10 seconds).
9	**frame_interval_ms**	Group CIRS into one frame. One frame has sensing and idle parts. *Used if sensing_mode is 1.*
10	**num_cir_per_frame**	Number of CIRS in a single frame. Maximum is 32. *Used if sensing_mode is 1.*
11	**deep_sleep_en**	**1** or **0**. Puts the SoC in deep-sleep during the idle part. *Used if sensing_mode is 1.*
12	**num_rx_mode**	**1** - to return Rx2 CIR only**, 2**- to return both Rx1 and Rx2
13	**preamble_index**	Preamble code selection **1-8** LG4A **9-24** BPRF **25-32** HPRF

© ChipsBank 2026

2.1 Mode_PRF

The radar driver supports 3 modes with different pulse repetition frequencies – BPRF, HPRF, and legacy LG4A.

Table 2. Different PRF modes used in radar sensing

mode_PRF	Mode	Packet Frame Length	Transmit Power Range	Mean PRF
0	**BPRF**	73.2us (127x4x2ns)x72	-55 to -25 dBm/MHz	62.89 MHz
1	**HPRF**	54.5us (91x4x2ns)x72	-52.5 to -22.5 dBm/MHz	111.09 MHz
2	**LG4A**	71.4us (31x16x2ns)x72	-61 to -31 dBm/MHz	16.1 MHz

2.2 Power Code, Gain Index and Scale Bit

The power code refers to the transmitter power amplifier gain that controls the output power. Every step in the power code corresponds to around 0.5 dB. The gain index controls the receiver gain with each increment equal to approximately -6 dB. The scale bit is a pre-scaler in the baseband processing.

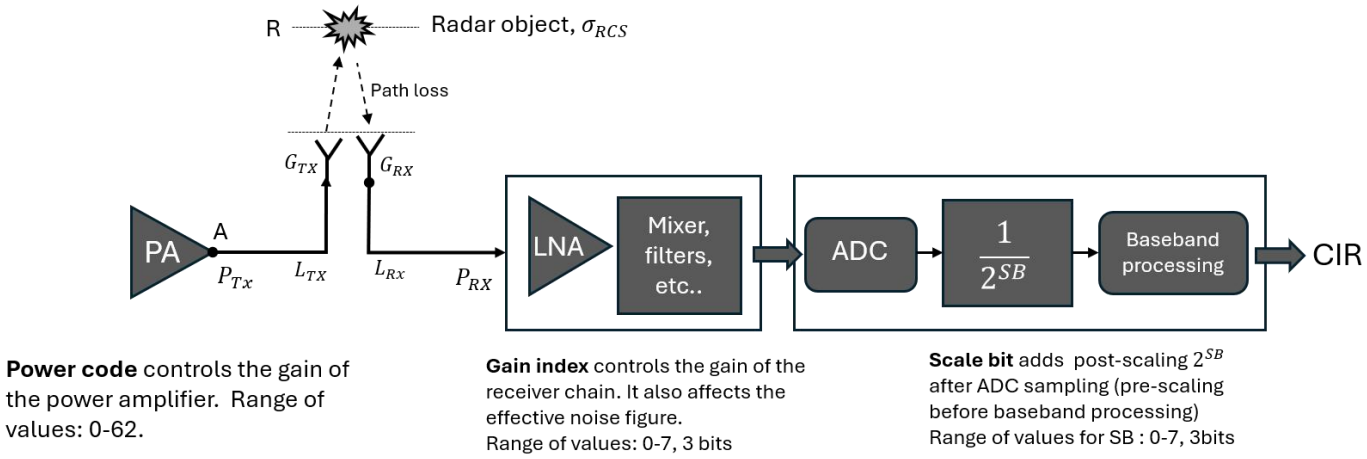


Figure 2-1. Illustration for the components being controlled by the power code, gain index and scale bit.

Table 3. Power code and corresponding power density

Power code	Power @ Chip Port BPRF (dBm/MHz)	Power @ Chip Port HPRF (dBm/MHz)	Power @ Chip Port LG4A (dBm/MHz)
0	-55	-52.5	-61
1	-54.5	-52	-60.5
2	-54	-51.5	-60
3	-53.5	-51	-59.5
8-12		Unused	
58	-26	-23.5	-20
59	-25.5	-23	-19.5
60	-25	-22.5	-19

© ChipsBank 2026

Table 4. Gain index and the corresponding gain, noise figure and input power saturation point

Gain Index	LNA Gain (dB)	Noise Figure (dB)	Input Saturation Point (dBm, BPRF)	Input Saturation Point (dBm, HPRF)	Input Saturation Point (dBm, LG4A)
0	60	4.0	-70	-67.5	-76
1	55	4.1	-65	-62.5	-71
2	50	4.4	-60	-57.5	-66
3	44	5.8	-54	-51.5	-60
4	37	8.6	-48	-45.5	-54
5	31	12.1	-41	-38.5	-47
6	23	20.0	-33	-30.5	-39
7	12	28.9	-22	-19.5	-28

2.3 CIR_tap_start and CIR_tap_end

The **CIR_tap_start** and **CIR_tap_end** refers to the first and last index of the requested CIR IQ data. **Tap index = 30** is the approximate location of R=0 m, estimated from the SMA port of the evaluation kit. Tap index = 0 corresponds to some *negative* distance.

2.4 Continuous sensing (sensing_mode = 0)

When **sensing_mode** is set to 0, the radar is continuously transmitting and acquiring CIR data. Each CIR will be available at regular intervals equal to the **sensing_interval_ms** provided. This mode is mainly used for simultaneous sensing and processing.

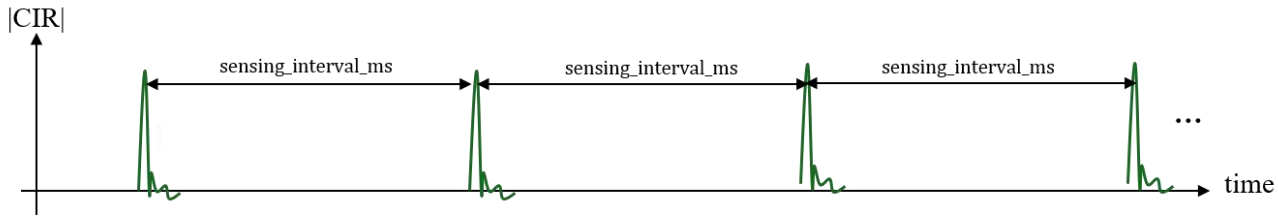


Figure 2-2. Continuous sensing mode

2.5 Burst mode sensing_burst_num

The test app also allows for burst mode wherein several CIRs can be combined by averaging. This is enabled when **sensing_burst_num** is set to a value greater than 1. This effectively maintains the signal magnitude and reduces the noise floor (increase in signal-to-noise-ratio).

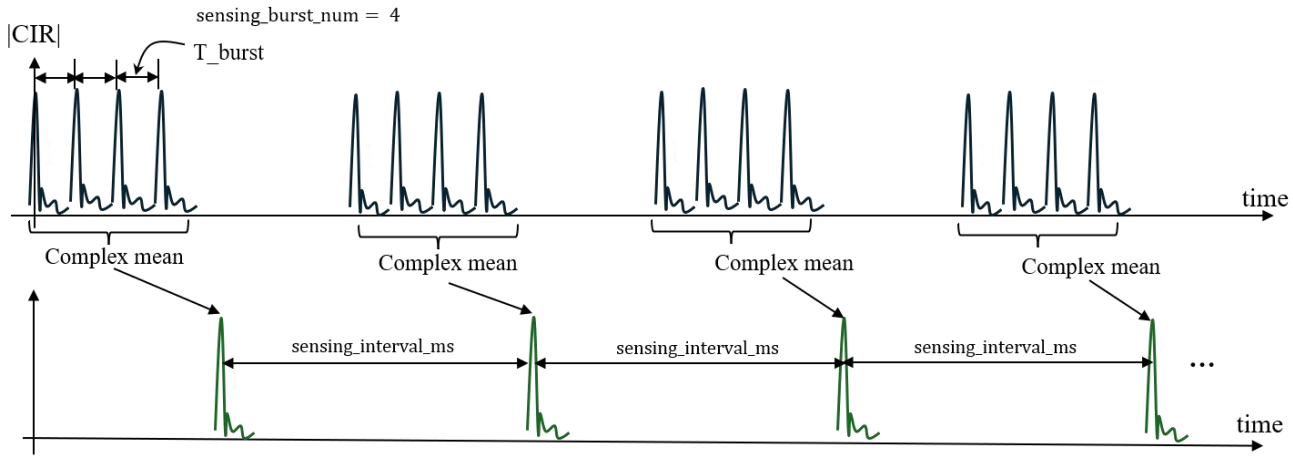


Figure 2-3. Continuous sensing mode with burst.

In the burst mode, the inter-burst time T_burst changes with the number of receivers and length of taps need to be returned. Approximately,

$$\text{T_burst_us} \approx 104 + (28 + 0.57 \cdot N) \cdot \text{num_rx_mode}$$

where N is the number of CIR taps $(\text{CIR}_{\text{tap_end}} - \text{CIR}_{\text{tap_start}} + 1)$ and T_burst is in μs.

Note that the total burst time must not exceed the desired sensing interval:

$$\text{sensing_interval_ms} > \text{sensing_burst_num} \cdot \left(\frac{\text{T_burst_us}}{1000}\right).$$

.

Table 5. Maximum number of sensing_burst_num for num_rx_mode=1

Number of Taps	sensing_burst_num (max)
$N \leq 17$	7 * sensing_interval_ms
$18 \leq N \leq 59$	6* sensing_interval_ms
$60 \leq N \leq 119$	5* sensing_interval_ms
$120 \leq N \leq 207$	4* sensing_interval_ms
$208 \leq N \leq 241$	3 * sensing_interval_ms

Table 6. Maximum number of sensing_burst_num for num_rx_mode=2

Number of Taps	sensing_burst_num (max)
$0 < N \leq 5$	6 * sensing_interval_ms
$6 \leq N \leq 35$	5* sensing_interval_ms
$36 \leq N \leq 78$	4* sensing_interval_ms
$79 \leq N \leq 151$	3* sensing_interval_ms
$152 \leq N \leq 241$	2* sensing_interval_ms

2.6 Frame-based sensing (sensing_mode = 1)

In frame-based sensing, several CIRs are collected to form a single radar frame. This is enabled by setting the **sensing_mode** = **1**. If enabled, frame-based parameters **frame_interval_ms, num_cir_per_frame** are used. The **sensing_interval_ms** and **sensing_burst_num** are also applicable in this mode. If needed, the driver is also capable of deep sleep by enabling **deep_sleep_en** and is only used in this frame-based mode.

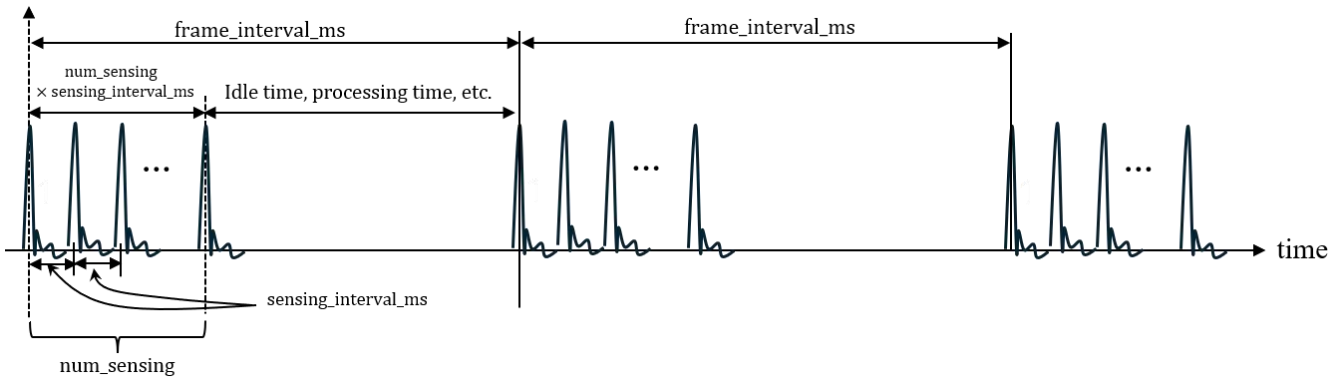


Figure 2-4. Frame-based mode. Burst mode can be enabled but not shown in this figure.

3 Test APP Data Format

One purpose of the test app is to return the CIR data via UART for algorithm development. This section gives details on the timing diagram describing the sequence from data acquisition to CIR data printing. Information about the binary format is also provided to facilitate parsing.

3.1 Sensing mode = 0 (continuous sensing)

In sensing mode = 0, the CIR data is sent after one CIR data is available. An example of parameters settings is shown in Table 7. The corresponding timing diagram is shown in Figure 3-1.

Table 7. An example of parameter settings for continuous mode

idx	Radar Config Parameter	Value
0	**mode_PRF**	**0** (BPRF)
1	**power_code**	**48**
2	**scale_bit**	**6**
3	**gain_idx**	**2**
4	**CIR_tap_start**	**30** (R=0 m)
5	**CIR_tap_end**	**70** (R=6m)
6	**sensing_mode**	**0** (continuous)
7	**sensing_burst_num**	**4**
8	**sensing_interval_ms**	**5**
9	**frame_interval_ms**	**100** (don't care since sensing_mode = 0)
10	**num_cir_per_frame**	**32** (don't care since sensing_mode = 0)
11	**deep_sleep_en**	**0** (don't care since sensing_mode = 0)
12	**num_rx_mode**	**2** (Rx1 and Rx2)
13	**preamble_index**	**9**
UART Command	**f,0,48,6,2,30,70,0,4,5,100,32,0,2,9**	

© ChipsBank 2026

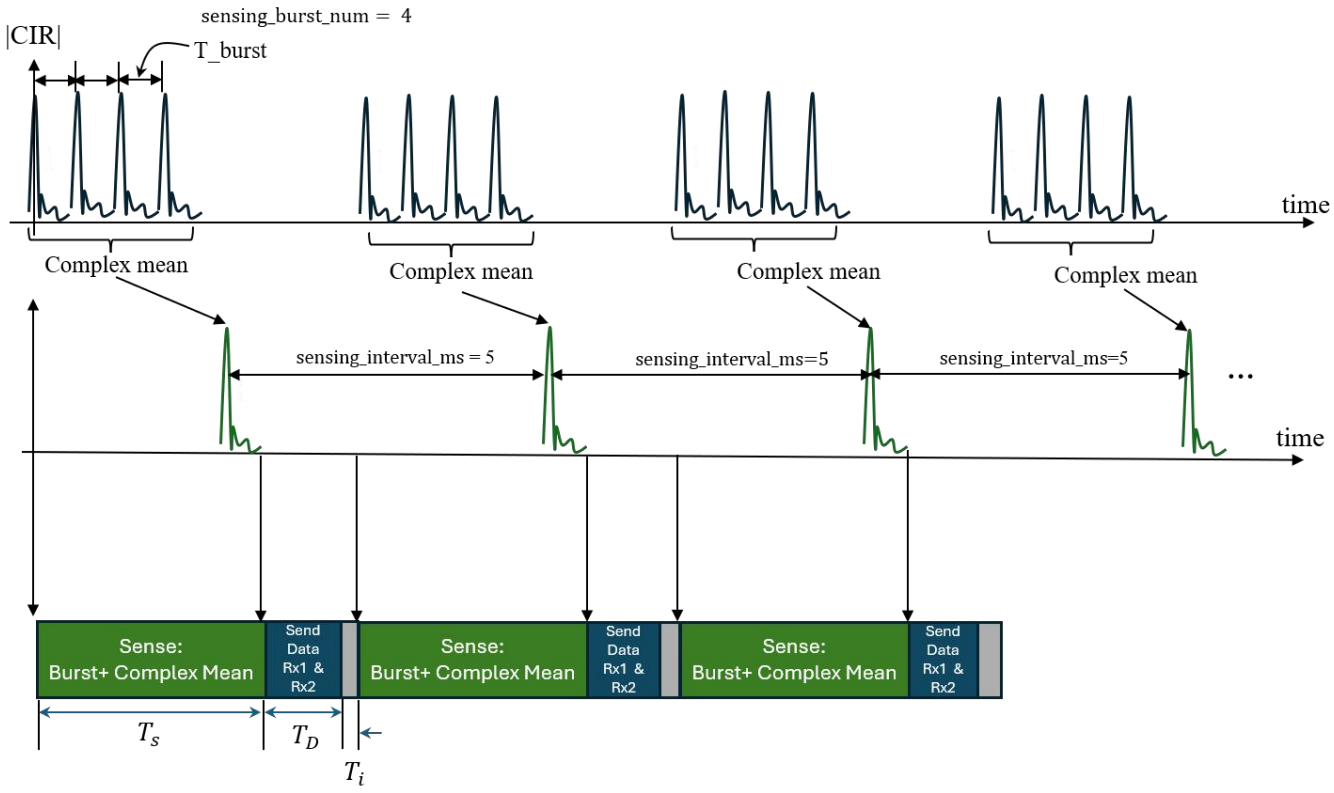


Figure 3-1. Test app timing diagram for continuous mode. T_S, T_D and T_i are the sensing, data transfer and idle time for each CIR cycle, respectively.

It requires T_S to complete all the bursts and the subsequent averaging.

$$T_S(\text{ms}) = \text{sensing_burst_num} \cdot \left(\frac{\text{T_burst_us}}{1000}\right).$$

The UART printout requires $33\ \mu s$ for one tap (32- bit complex) per receiver at 921600 baud rate.

$$T_D(\text{ms}) \cong \frac{0.033\ \text{ms}}{\text{tap} \cdot \text{receiver}} \cdot \left(\text{CIR}_{\text{tap_end}} - \text{CIR}_{\text{tap_start}} + 1\right) \cdot \text{num_rx_mode}$$

For long CIR sequence, the sensing interval must be sufficiently long to allow the data to be sent via UART.

$$\text{sensing_interval_ms} > T_S + T_D$$

3.2 *Sensing mode = 1 (frame-based sensing)*

For frame-based sensing, the data is sent after every frame. An example of parameter settings is shown in Table 8. The corresponding timing diagram are shown in Figure 3-2.

Table 8. An example of parameter settings for frame-based mode

idx	Radar Config Parameter, P[idx]	Value
0	**mode_PRF**	**0** (BPRF)
1	**power_code**	**48**
2	**scale_bit**	**6**
3	**gain_idx**	**2**
4	**CIR_tap_start**	**30** (R=0 m)
5	**CIR_tap_end**	**70** (R=6m)

6	**sensing_mode**	**1** (frame-based)
7	**sensing_burst_num**	**1**
8	**sensing_interval_ms**	**1**
9	**frame_interval_ms**	**100**
10	**num_cir_per_frame**	**32**
11	**deep_sleep_en**	**0**
12	**num_rx_mode**	**2**
13	**preamble_index**	**9**
UART Command	**f,0,48,6,2,30,70,1,1,1,100,32,0,2,9**	

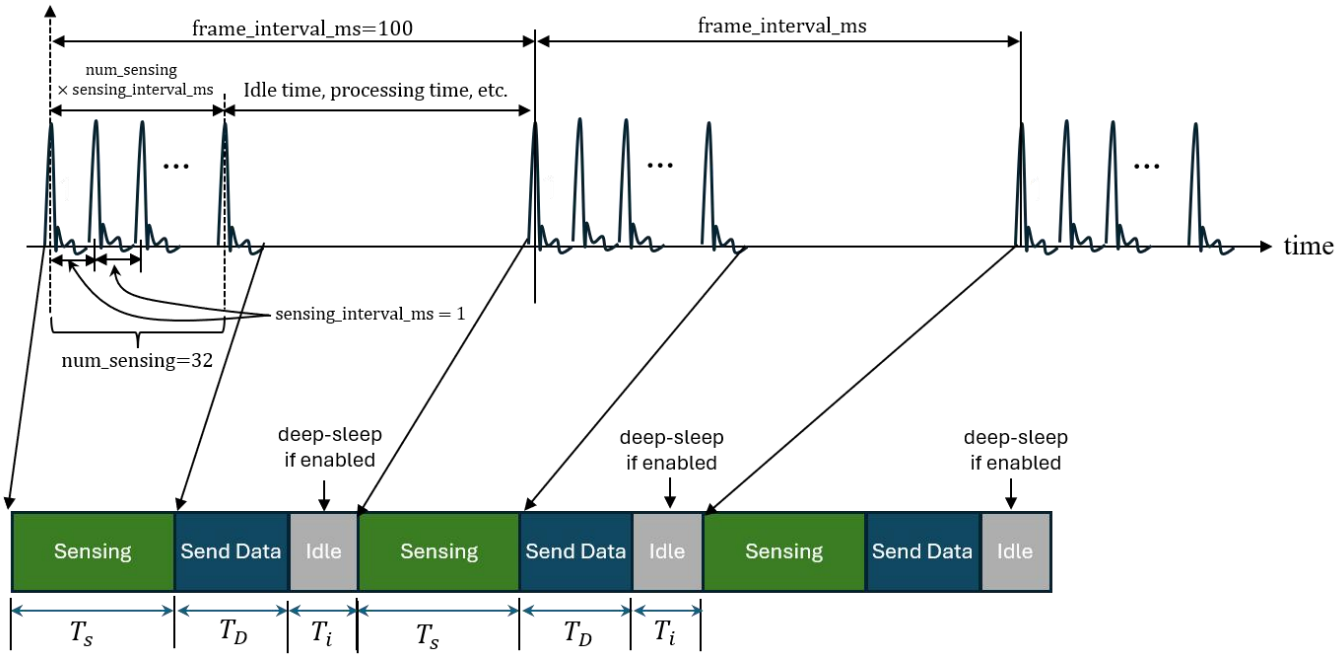


Figure 3-2. Test app timing diagram for frame-based mode. T_S, T_D and T_i are the sensing, data transfer and idle time for one complete frame, respectively.

For long CIR sequence, the frame interval must be sufficiently long to allow the data to be sent via UART at 921600 baud rate. Note that the data is sent only after one frame is complete. The sensing and data transfer time are calculated as follows:

$$T_S\ (\text{ms}) \cong \text{num_sensing} \times \text{sensing_interval_ms}$$

$$T_D(\text{ms}) \cong \frac{0.033\ \text{ms}}{\text{tap} \cdot \text{receiver}} \cdot \left(\text{CIR}_{\text{tap_end}} - \text{CIR}_{\text{tap_start}} + 1\right) \cdot \text{num_sensing} \cdot \text{num_rx_mode}$$

$$\text{frame_interval_ms} > T_s + T_D$$

Note that burst-mode is still applicable in this sensing mode, hence the sensing interval should still satisfy:

$$\text{sensing_interval_ms} > \text{sensing_burst_num} \cdot \left(\frac{\text{T_burst_us}}{1000}\right).$$

© ChipsBank 2026

3.3 Logging in binary format

The test app prints the headers in ASCII followed by a binary stream of the IQ data. An example shown in Figure 3-1 is the CIR data returned by the test app via UART.

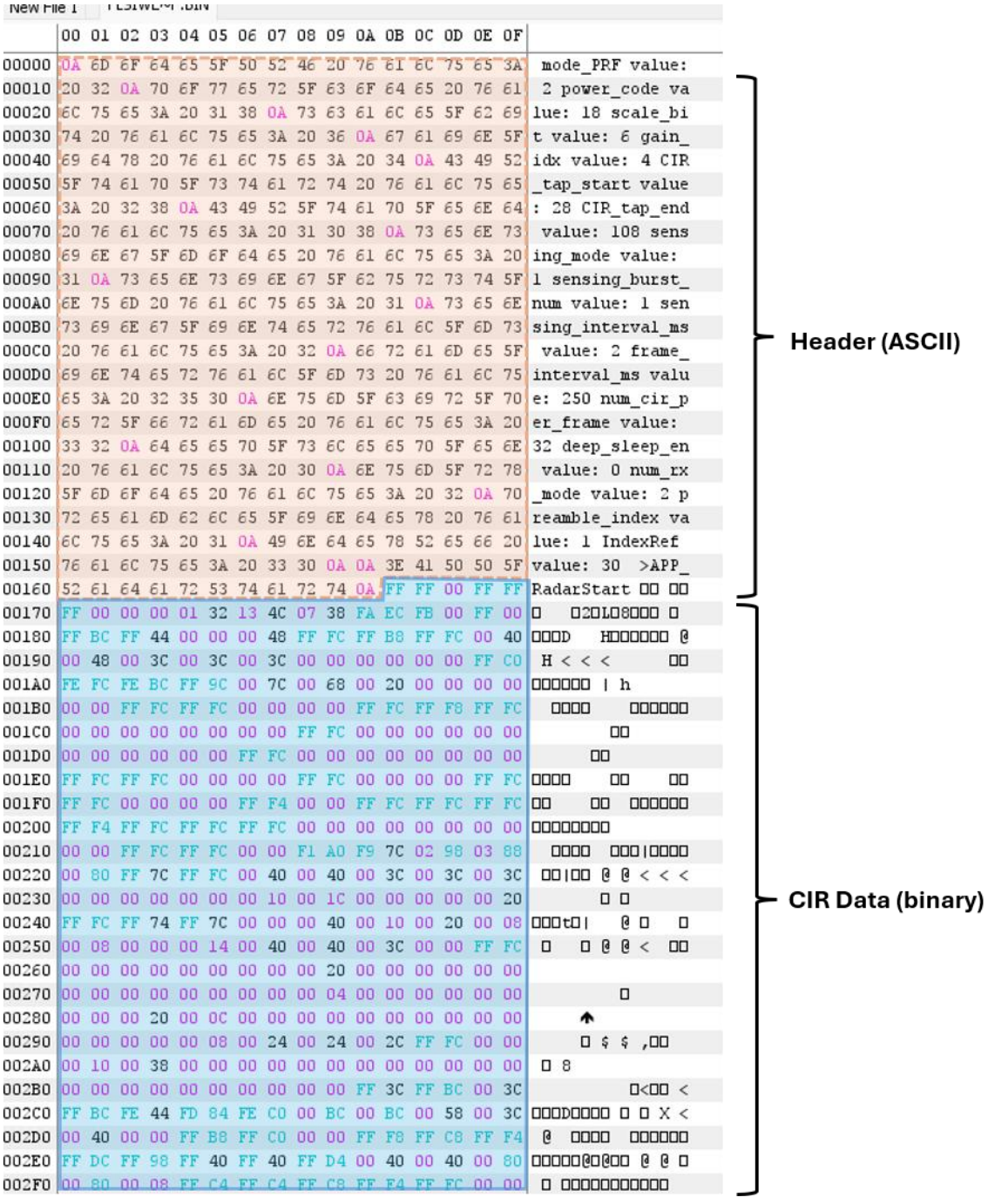


Figure 3-3. CIR data returned by the test App via UART.

The header is immediately followed by the CIR data. The CIR data is composed of the following:

- num_rx_mode = 1: Identifier, Sequence and Frame ID, Rx2 Data

FF FF 00FF FFFF00	XX XX XX	YY	I2 ….	Q2 …
Identifier 7 bytes	Sequence ID 3 bytes	Frame ID 1 byte	Rx2 I and Q data I: N * 2bytes Q: N * 2bytes N is number of taps	

- num_rx_mode = 2: Identifier, Sequence and Frame ID, Rx1 Data, Rx2 Data

FF FF 00FF FFFF00	XX XX XX	YY	I1 ….	Q1 …	I2 ….	Q2 …
Identifier 7 bytes	Sequence ID 3 bytes	Frame ID 1 byte	Rx1 I and Q data I: N * 2bytes Q: N * 2bytes N is number of taps		Rx2 I and Q data I: N * 2bytes Q: N * 2bytes N is number of taps	

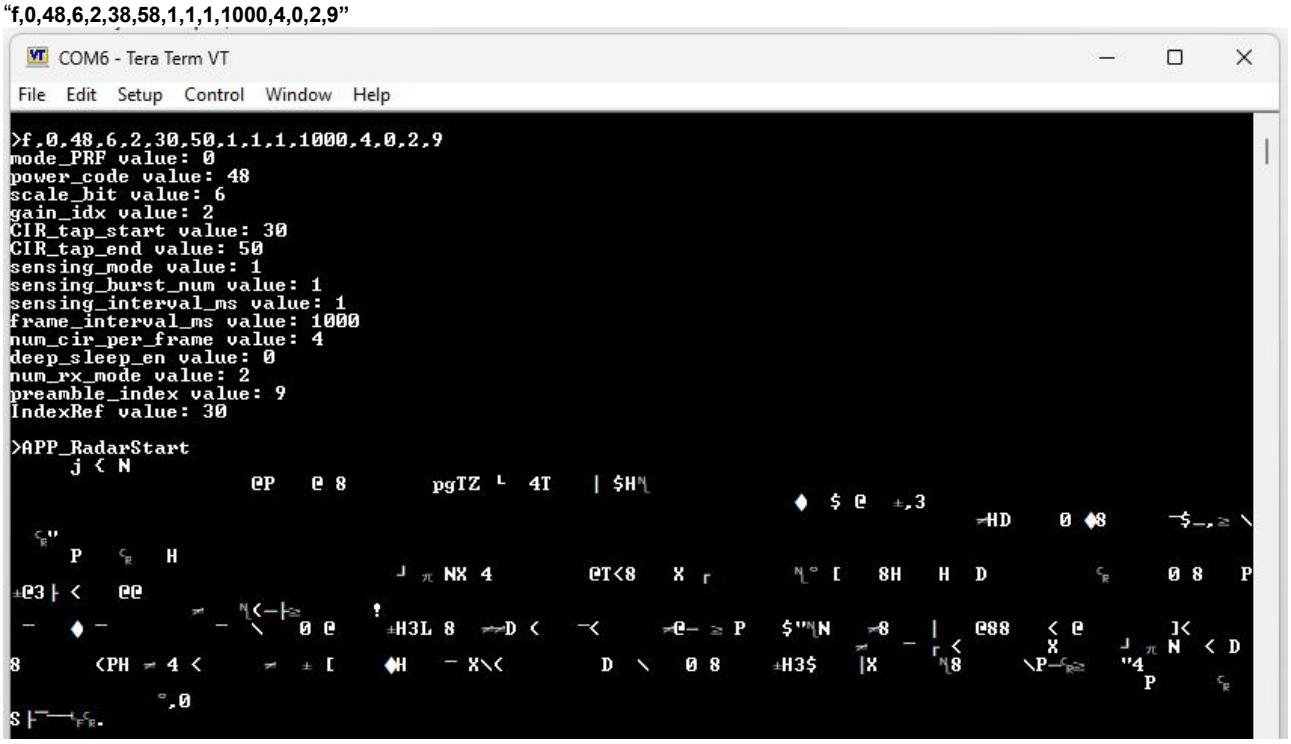
For continuous mode (sensing_mode=0), only the sequence ID is incremented for every CIR printed. For frame-based mode (sensing_mode=1), the frame ID is incremented whenever a new frame is printed. The sequence ID is incremented for each CIR and CIRs within the same frame have the same frame ID. The sequence ID is not being reset for each new frame.

3.4 Test App Command

Primarily, the command-line interface version of the test app accepts two commands:

- Command to start the radar logging requires 14 parameters in the sequence as indicated in Table 1. For example:

  "f,0,48,6,2,38,58,1,1,1,1000,4,0,2,9"



- Command to stop the acquisition

  "f,0"

- **IMPORTANT** For deep sleep mode, "f,0" has to be input repeatedly and continuously as the command has to be sent while the SoC is 'awake'. Alternatively, a GPIO (GPIO 7 in the release) can be set to low to prevent the hardware from going into deep-sleep – then followed by the stop "f,0" command.

© ChipsBank 2026